Department of Computer Science and Engineering
Assignment Questions
Course Code & Name: CSA0312 – Data Structures – Slot B

Assignment Title:	Design Intelligent Emergency Evacuation Routing System for a city containing more than 8000 intersections and 20,000 weighted roads
Course Outcome(s) – CO:	**CO2:** Apply the suitable Abstract Data type for construction of the smart city emergency system. **CO4:** Make use of heaps, priority queue and hashing techniques for graph storage and route cache for repeated queries in C. **CO5:** Develop robust smart city emergency system by implementing shortest path algorithms such as Dijkstra's algorithm, Floyd-Warshall.
Bloom's Taxonomy Level	L4 – Analyze
SDG Mapping (SDG 1-17)	SDG 9 - Industry, Innovation, and Infrastructure SDG 11 – Sustainable Cities and Communities; SDG 13 – Climate Action

Problem Statement:

A smart-city disaster management authority is developing an Intelligent Emergency Evacuation Routing System for a city containing more than 8000 intersections and 20,000 weighted roads. During floods, earthquakes, industrial accidents, or large-scale fires, road travel costs change dynamically because of congestion, road blockages, damaged infrastructure, and emergency restrictions.

The system receives continuous updates containing intersections, roads, travel costs, road status, evacuation centers, and emergency vehicle locations. It must identify the safest and shortest available route from multiple disaster locations to designated evacuation centers while operating under the following constraints:

- The network contains 8000+ vertices and 20,000+ weighted edges.
- Road weights change frequently because of disaster conditions.
- Blocked roads must be removed or temporarily disabled.
- The system should support multiple source-to-destination queries.

- Repeated evacuation queries should be answered efficiently.
- Memory consumption should remain within moderate computational resources.
- The system should avoid repeatedly processing the same intersections.
- The routing system should prioritize routes with minimum travel cost.
- The system should identify unreachable evacuation centers.
- The implementation must be developed in C.

The engineering team is considering Adjacency Matrix, HashMap-based Adjacency List, HashSet, Min-Heap, Priority Queue, Dijkstra's Algorithm, Floyd-Warshall, and route caching.

Students must:

1. Analyze and justify the choice of graph ADT and graph representation.
2. Design suitable C structures for vertices, edges, hash entries, heap nodes, and routes.
3. Implement dynamic road-weight and road-status updates.
4. Use a Min-Heap to optimize Dijkstra's algorithm.
5. Use a HashSet to track processed intersections.
6. Implement route reconstruction.
7. Implement route caching for repeated queries.
8. Handle blocked roads, invalid vertices, duplicate roads, and unreachable destinations.
9. Compare Dijkstra's algorithm with Floyd-Warshall for the given network.
10. Analyze the time and space complexity of the complete system.
11. Evaluate whether the design can satisfy a 200 ms routing requirement.
12. Analyze how the system behaves when 30% of the roads become unavailable during a disaster

ANSWER

Student Name	Register Number	Team Members	Date
Athisaya U	192571001	MONIKA B (192571002) PAPAGARI PUSHPA CHARITHA (192525371)	31/08/2026

GitHub Repository: https://github.com/athisayaprema2007-hue/CSA0312-Intelligent-Evacuation-Routing-System

1. Graph ADT and Graph Representation

Problem Understanding

The road system is naturally represented as a weighted, undirected graph. Every intersection is a vertex, while every bidirectional road is stored as an edge with a non-negative safety-adjusted travel cost. The graph must support frequent weight changes, road blocking and reopening, multiple source-to-destination requests, repeated requests, route reconstruction and unreachable-destination reporting.

Selected Representation

A HashMap-based adjacency list is selected. With 8,200 intersections, the number of possible undirected pairs exceeds 33 million, whereas the test network contains only 21,000 roads. The network is therefore sparse. An adjacency matrix would reserve O(V^2) cells, most of which would be unused. The adjacency list stores only existing roads in O(V+E) space and allows Dijkstra to scan only the neighbours of the current intersection. The HashMap maps arbitrary intersection IDs to Vertex records in average O(1) time.

Representation	Time / Space Effect	Decision
Adjacency Matrix	O(V^2) memory; O(V) neighbour scan	Rejected
HashMap Adjacency List	O(V+E) memory; O(degree) neighbour scan	Selected
Floyd-Warshall Matrix	O(V^3) preprocessing and O(V^2) memory	Rejected at full scale

2. C Structure Design

The structures are separated into modules so each ADT has one responsibility. External IDs remain arbitrary integers, while a dense index field supports efficient arrays such as dist[], prev[] and heap pos[].

Structure	Important Fields	Purpose
Edge	dest_id, weight, active, next	One directed half of a road
Vertex	id, index, edges, degree	Intersection and adjacency-list head
Graph	vertices, by_index, counts, version	Complete mutable road network
HashMapEntry	key, value, next	ID-to-Vertex lookup
HashSetEntry	key, next	Processed-intersection membership
HeapNode / MinHeap	vertex_index, dist, nodes, pos	Priority queue and decrease-key
Route	IDs, length, total_cost, reachable	Reconstructed query result
RouteCacheEntry	src, dst, graph_version, route	Versioned repeated-query result

```
typedef struct Edge {
    int dest_id;
    long weight;
    int active;
    struct Edge *next;
} Edge;

typedef struct Route {
    int src_id, dst_id;
    int *intersection_ids;
    int num_intersections;
    long total_cost;
    int reachable;
} Route;
```

3. Dynamic Road-Weight and Road-Status Updates

Each logical road is stored as two directed Edge records. graph_update_road_weight validates both endpoints, rejects a negative value, finds both edge records and changes both weights together.

graph_set_road_status changes both active flags. A blocked road remains in the list with active = 0, preserving its weight for later reopening. Every routing-relevant mutation increments Graph.version because the best route may have changed.

```
UpdateRoadWeight(G, u, v, w):
    if u or v is invalid: return INVALID_VERTEX
    if w < 0: return NEGATIVE_WEIGHT
    locate edge u->v and edge v->u
    if either edge is absent: return NO_SUCH_ROAD
    update both weights
    G.version = G.version + 1
    return OK

SetRoadStatus(G, u, v, active):
    validate endpoints and locate both edge records
    set both active flags
    G.version = G.version + 1
    return OK
```

4. Min-Heap Optimisation of Dijkstra

The priority queue is an array-based binary min-heap ordered by tentative distance. Insert appends a node and sifts upward. Extract-min removes the root, moves the last node to the root and heapifies downward. Decrease-key lowers the distance of a vertex already in the heap and sifts it upward. A reverse pos[vertex_index] array locates each heap node in O(1), making decrease-key O(log V). Every heap swap updates both reverse positions.

```
ExtractMin(H):
    minimum = H.nodes[0]
    move H.nodes[H.size - 1] to the root
    reduce H.size and repair pos[]
    Heapify(H, 0)
    return minimum
```

```
DecreaseKey(H, vertex, new_distance):
    i = H.pos[vertex]
    reject an absent vertex or a non-decreasing value
    H.nodes[i].dist = new_distance
    SiftUp(H, i)
```

5. HashSet for Processed Intersections

Dijkstra creates a HashSet named processed for every search. After extract-min, the algorithm checks whether the vertex was already finalized. A duplicate or stale heap item is skipped; otherwise the vertex is inserted before its outgoing edges are relaxed. Separate chaining and resizing keep add and contains operations O(1) on average. This mechanism ensures that cyclic road networks cannot cause repeated finalization of the same intersection.

6. Dijkstra Algorithm and Route Reconstruction

The source and destination are resolved through the graph HashMap. dist[] is initialized to infinity, prev[] to NONE and the source distance to zero. The source is inserted into the heap. The algorithm repeatedly extracts the smallest tentative distance, finalizes that vertex using the HashSet, and relaxes only active outgoing roads. With non-negative weights, the first extraction of the destination proves its distance is optimal.

```
Dijkstra(G, src, dst):
    dist[*] = INF; prev[*] = NONE; dist[src] = 0
    insert src into the MinHeap
    processed = empty HashSet
    while heap is not empty:
        u = ExtractMin(heap)
        if u is in processed: continue
        add u to processed
        if u == dst: break
        for every active edge (u, v, w):
            if v is not processed and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                decrease-key or insert v
    return dist and prev
```

Route Reconstruction

If dist[dst] remains infinity, the function returns a valid Route with reachable = 0 and total_cost = -1. Otherwise the predecessor chain is followed backwards from the destination to count the number of intersections. A route array is allocated and filled in reverse so that the final order is source to destination.

```
ReconstructRoute(src, dst, dist, prev):
    if dist[dst] == INF: return UNREACHABLE
    count nodes while following prev[] from dst
    allocate route IDs
    fill IDs from the last index down to zero
    return route with total_cost = dist[dst]
```

7. Route Caching for Repeated Queries

RouteCache is a hash table keyed by the ordered pair (source, destination). Each entry stores a deep copy of the Route and the graph version that produced it. A matching version is a cache hit and returns in average O(1) time. A different version means that a road changed; the stale entry is removed lazily and the query is recomputed. Unreachable results are cached as well, preventing repeated expensive searches for the same disconnected destination.

```
EvacQuery(G, cache, src, dst):
    validate source and destination
    route = CacheLookup(cache, src, dst, G.version)
    if route exists: return route
    route = Dijkstra plus route reconstruction
    CacheStore(cache, route, G.version)
    return route
```

8. Exceptional and Failure Cases

Condition	Required Behaviour	Verification
Invalid source or destination	Return NULL / INVALID before search	Tests 6 and 7
Duplicate road	Reject without changing the graph	Test 8
Negative weight	Return GRAPH_ERR_NEGATIVE_WEIGHT	Test 9
Blocked road	Keep the edge; skip active = 0	Test 4

Reopened road	Restore traversal using stored weight	Test 5
Missing road	Return GRAPH_ERR_NO_SUCH_ROAD	Graph module
Unreachable destination	reachable = 0 and cost = -1	Tests 10 and 16
Cyclic graph	HashSet finalizes each vertex once	Test 15

Every allocation is checked. Cleanup functions release graphs, maps, sets, heaps, routes and cache entries. Partial road insertion is rolled back if the second directed edge cannot be allocated.

9. Dijkstra versus Floyd-Warshall

Aspect	Dijkstra with Binary Heap	Floyd-Warshall
Problem solved	Single-source / single-pair	All-pairs
Time	O((V+E) log V) per query	O(V^3)
Space	O(V+E) graph + O(V) working	O(V^2) matrix
Sparse graph	Uses only existing edges	Ignores sparsity
Road update	Next query reads the new graph	Matrix becomes stale
Repeated query	Average O(1) cache hit	O(1) only after costly preprocessing
Decision	Selected	Unsuitable at full scale

At V = 8,200, Floyd-Warshall requires approximately 5.5 x 10^11 updates and about 67.2 million distance cells. Its O(V^3) computation would also need to be repeated after important road changes. Dijkstra is therefore the appropriate choice for this sparse, dynamic network.

10. Time and Space Complexity

Operation	Time Complexity	Space Complexity
HashMap / HashSet lookup	O(1) average	O(V) total
Add intersection	O(1) average	O(1) additional
Add / update / block road	O(degree)	O(1) additional
Heap insert / extract / decrease-key	O(log V)	O(V) heap total
Uncached Dijkstra	O((V+E) log V)	O(V) working
Route reconstruction	O(L)	O(L)
Cache lookup / store	O(1) average	O(C x average L)
Complete graph storage	-	O(V+E)

V denotes intersections, E logical roads, L the route length and C cached routes. The complete system uses O(V+E+C x average L) memory, which is suitable for moderate computational resources.

11. Evaluation of the 200 ms Requirement

The deterministic benchmark uses 8,200 intersections and 21,000 logical bidirectional roads, represented

internally by 42,000 directed edge records. Ten fixed source-to-destination queries were executed. The measured uncached average was approximately 3.91 ms per query and the worst observed query was 7.000 ms. Cached hits averaged approximately 0.00003 ms.

The recorded configuration was Windows 11 on an Intel Core Ultra 7 255H system with 31.4 GB usable RAM, using GCC 3.4.2 with -O2 optimisation.

Metric	Measured Result
Intersections	8,200
Logical roads	21,000
Average uncached query	Approximately 3.91 ms
Worst uncached query	7.000 ms
Average cached hit	Approximately 0.00003 ms
Functional tests	16 / 16 PASS

The measured configuration therefore satisfies the 200 ms routing requirement with substantial margin. The conclusion is limited to the stated machine, compiler, deterministic dataset and clock resolution; it is not claimed as a universal hardware guarantee.

12. Analysis When 30% of Roads Become Unavailable

The failure experiment blocks exactly 6,300 of the 21,000 logical roads using a deterministic shuffle. All ten selected source-destination queries remain reachable, but global reachability from the reference intersection falls from 8,200/8,200 to 8,026/8,200. Therefore 174 intersections, approximately 2.1%, become disconnected from the main reachable region.

Observation	Before Failure	After 30% Failure
Open logical roads	21,000	14,700
Globally reachable intersections	8,200	8,026
Isolated from reference source	0	174
Selected destinations reachable	10 / 10	10 / 10
Cached routes invalidated	0	10
Route-cost change	Baseline	+3 to +175

Route lengths usually increase because blocked segments force detours. Runtime remains in the same few-millisecond range because the algorithm still scans adjacency lists and checks the active flag. The graph version changes after blocking, so all ten cached routes are correctly recognized as stale and recomputed. The experiment shows graceful degradation: most of the city remains routable, while isolated neighbourhoods are explicitly identified.

Test Cases and Results

Test Group	Coverage	Result
Shortest route	Hand-verified path and cost	PASS
Dynamic graph	Weight update, block and reopen	PASS
Validation	Invalid IDs, duplicates and negative weights	PASS
Reachability	Disconnected and isolated vertices	PASS
Repeated queries	Cache miss, hit and invalidation	PASS
Cycles	No repeated finalization	PASS
Overall	16 functional tests	16 / 16 PASS

Reflection and SDG Relevance

The implementation demonstrates that the supporting ADTs determine whether Dijkstra scales in practice. The adjacency list avoids matrix waste, the binary heap avoids O(V) frontier scans, the HashSet prevents repeated processing, and the version-stamped cache makes invalidation reliable. Keeping blocked roads as inactive records makes blocking and reopening symmetric and prevents data loss.

- SDG 9 - Industry, Innovation and Infrastructure: the system is resilient infrastructure software that remains responsive as road conditions change.
- **SDG 11:** This project connects to safer city planning because it shows how an evacuation system can reroute people when roads are blocked and can identify areas that become unreachable.
- SDG 13 - Climate Action: adaptive routing is a practical response to road failures caused by floods, storms, fires and other climate-related emergencies.

Individual Technical Analysis

I analysed the implementation as a dynamic shortest-path system rather than as a static map. The key design decision was selecting a HashMap-based adjacency list because the city is sparse: storing 21,000 logical roads is substantially more suitable than reserving an adjacency matrix for every possible pair of 8,200 intersections. I also traced how every road update changes Graph.version, making an older cached route stale and forcing a fresh Dijkstra computation when needed.

The observed outputs support the design decisions. The test run confirms validation, route reconstruction, cache invalidation and cyclic-graph handling. The benchmark confirms that the binary heap keeps route searches far below the 200 ms requirement. Finally, the 30% road-failure result shows a realistic trade-off: selected evacuation centres remain reachable, while disconnected intersections are explicitly reported instead of producing an incorrect route.

Verified Execution Evidence

The following verified console-output excerpts are taken from the Windows execution of the program. Timing values can vary slightly between runs because of clock resolution, but all uncached queries remain far below the required 200 ms limit.

```
[8] Multiple queries from different disaster sites.
Query 2 -> 7 [cache MISS, Dijkstra run]: Route: 2 -> 4 -> 5 -> 6 -> 7  (total travel cost 11, 5 intersections)
Query 3 -> 6 [cache MISS, Dijkstra run]: Route: 3 -> 1 -> 2 -> 4 -> 5 -> 6  (total travel cost 15, 6 intersections)
Query 7 -> 1 [cache MISS, Dijkstra run]: Route: 7 -> 6 -> 5 -> 4 -> 2 -> 1  (total travel cost 15, 6 intersections)

[9] Unreachable center: intersection 8 has no roads.
Query 1 -> 8 [cache MISS, Dijkstra run]: Destination 8 is UNREACHABLE from 1 over active roads.

[10] Invalid queries.
Query 99 -> 4 [cache MISS, Dijkstra run]: rejected: invalid intersection ID
Query 1 -> -5 [cache MISS, Dijkstra run]: rejected: invalid intersection ID

[11] Cache statistics: hits=1 misses=8 stale_evictions=3

Demo finished; freeing all memory.

test_runner: 16/16 tests passed
```

Figure 1. Console-output excerpt showing multiple routes, unreachable and invalid-query handling, cache statistics, and 16/16 functional tests passed.

```
Generating deterministic city (seed 20260831): 8200 intersections, 21000 logical roads...
Graph ready: 8200 vertices, 21000 logical roads (42000 directed edges).
Query  1:  4546 ->  2343  cost=   161  hops=   9  uncached avg 1.000 ms  cached avg 0.000000 ms
Query  2:   488 ->  4320  cost=   154  hops=   5  uncached avg 0.000 ms  cached avg 0.000160 ms
Query  3:  7680 ->  4213  cost=   212  hops=   9  uncached avg 1.067 ms  cached avg 0.000000 ms
Query  4:  8005 ->  1947  cost=   223  hops=  11  uncached avg 1.067 ms  cached avg 0.000000 ms
Query  5:  5674 ->  1101  cost=   241  hops=   7  uncached avg 1.067 ms  cached avg 0.000000 ms
Query  6:  3568 ->  5188  cost=   215  hops=  12  uncached avg 3.133 ms  cached avg 0.000000 ms
Query  7:  7204 ->  7818  cost=   238  hops=  13  uncached avg 2.133 ms  cached avg 0.000000 ms
Query  8:  7116 ->  6482  cost=   228  hops=   9  uncached avg 2.933 ms  cached avg 0.000000 ms
Query  9:  4541 ->  3854  cost=   241  hops=  10  uncached avg 1.067 ms  cached avg 0.000000 ms
Query 10:  6012 ->  4594  cost=   265  hops=   7  uncached avg 2.133 ms  cached avg 0.000000 ms
Benchmark written to results\benchmark_results.csv
Cache stats: hits=1000000 misses=10 stale_evictions=0
```

Figure 2. Console-output excerpt from the full-scale benchmark with 8,200 intersections, 21,000 logical roads and ten timed route queries.

```
Building the same deterministic city as the benchmark...
Blocked 6300 of 21000 logical roads (30%).
Analysis written to results\road_failure_analysis.txt
```

Figure 3. Console-output excerpt from the road-failure experiment, where exactly 6,300 of 21,000 logical roads were blocked and the analysis file was written.

Implementation and Submission Evidence

The complete modular C implementation is provided in the accompanying project ZIP and GitHub repository. The repository contains src/, tests/, results/, build files, the full technical report and the rubric checklist. The main modules are graph.c, hashmap.c, hashset.c, minheap.c, dijkstra.c, route_cache.c and main.c. The recorded benchmark and road-failure results are generated by the supplied program rather than manually entered.

Repository: https://github.com/athisayaprema2007-hue/CSA0312-Intelligent-Evacuation-Routing-System

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & ADT Selection CO2	15	Clearly analyzes disaster-routing requirements and selects appropriate ADTs with strong justification	Most requirements identified with appropriate ADTs	Basic understanding of requirements	Limited understanding
Graph, HashMap & HashSet Design- CO2/CO4	20	Efficient graph representation with correct HashMap/HashSet integration	Correct implementation with minor issues	Basic implementation	Inefficient/incomplete design
Min-Heap & Priority Queue Integration- CO4	20	Correct insert, extract-min, heapify/decrease-key operations and efficient integration	Mostly correct implementation	Priority queue works but inefficient	No effective heap implementation
Dijkstra & Route Reconstruction- CO5	20	Correct shortest route, dynamic weights, blocked roads and route reconstruction	Correct with minor issues	Works for simple cases	Major algorithmic errors

Performance, Testing & Complexity Analysis- CO4/CO5	15	Comprehensive scalability, complexity and performance analysis	Good analysis with minor omissions	Basic testing/analysis	Poor or missing analysis
Reflection & SDG Relevance- CO2/CO5	10	Strong technical reflection and clear SDG 9, 11 and 13 connection	Good justification and SDG connection	Generic reflection	Missing/irrelevant reflection

Deliverables

To receive full credit, each submission must include the following deliverables:

- Pseudocode
- Complete modular C implementation
- Test cases and results
- Complexity analysis
- Dijkstra vs Floyd-Warshall comparison
- Performance analysis
- GitHub repository
- Reflection on design decisions and SDG relevance